INTRODUCTION

What arrays are?

Arrays are a fundamental data structure in programming that allow you to store and manipulate collections of data.

Importance of arrays in programming

Arrays provide an efficient way to manage and organize data, making it easier to work with large sets of information.

How arrays simplify the handling of data

Arrays simplify data handling by providing a structured and indexed approach to store and retrieve information.

Concept Illustration

```
C/C++
Multiple variables:
int v1 = 10;
```

```
C/C++
int v2 = 20;
```

```
C/C++
int v3 = 30;
```

```
C/C++
int v4 = 40;
```

-
```
C/C++
int v5 = 50;
```
-
```
C/C++
Instead → Single array:
int arr[] = {10, 20, 30, 40, 50};
```

FORMAL DEFINITION

"An array is a collection of elements, each identified by an index or a key."

SYNTAX OF ARRAY DECLARATION

```
C/C++
data_type array_name[size];
```

OR

```
C/C++
data_type array_name[size1][size2]...[sizeN];
```

Where **N is the number of dimensions**

ARRAY INITIALIZATION METHODS

1. Initializing at the time of declaration

```C/C++
// Method 1: Specify individual elements
```

```C/C++
int array1[] = {1, 2, 3, 4, 5};
```

```C/C++
// Method 2: Specify size and assign later
```

```C/C++
int array2[5];
```

```C/C++
array2[0] = 1;
```

```C/C++
array2[1] = 2;
```

```C/C++
array2[2] = 3;
```

```C/C++
array2[3] = 4;
```

```C/C++
array2[4] = 5;
```

---

2. Using a loop for initialization

```C/C++
// Method 3: Initialize using loop
```

```C/C++
int array3[5];
```

```C/C++
for (int i = 0; i < 5; ++i) {
```

```C/C++
    array3[i] = i + 1;
```

```C/C++
}
```

---

3. Using designated initializers (C99 and later)

```C/C++
// Method 4: Designated initializers
```

```C/C++
int array4[] = {[0] = 1, [1] = 2, [2] = 3, [3] = 4, [4] = 5};
```

---

4. Initializing with a constant value

```C/C++
// Method 6: Initialize all elements with 0
```

```C/C++
int array6[5] = {0};
```

ACCESSING ARRAY ELEMENTS

How to access elements

Arrays use **indexing**

Index starts from **0 (zero-based indexing in C)**

Example Program

C/C++
```
#include <stdio.h>
```

C/C++
```
int main() {
```

C/C++
```
    // Declare array
```

C/C++
```
    int numbers[] = {10, 20, 30, 40, 50};
```

C/C++
```
    // Find size of array
```

C/C++
```
    size_t size_of_array = sizeof(numbers) / sizeof(numbers[0]);
```

C/C++
```
    printf("Size of the array: %zu elements\n", size_of_array);
```

C/C++
```
    // Print addresses of elements
```

```C/C++
    printf("Addresses of individual elements:\n");
```

```C/C++
    for (size_t i = 0; i < size_of_array; ++i) {
```

```C/C++
        printf("Element at index %zu has address: %p\n", i,
(void*)&numbers[i]);
```

```C/C++
    }
```

```C/C++
    return 0;
```

```C/C++
}
```

MULTIDIMENSIONAL ARRAYS

Topics Covered

Introduction to multidimensional arrays

Syntax and examples of 2D arrays

Concept

1D Array → Single row of elements

2D Array → Rows and columns

Example:

```C/C++
int arr[3][3];
```

STRINGS AS ARRAYS OF CHARACTERS

Key Points

Strings in C are **arrays of characters**

End with **null character '\0'**

Example

```C/C++
char str[] = "HOME";
```

Representation

```C/C++
Index:   0   1   2   3   4
```

```C/C++
Value:   H   O   M   E  \0
```

STRING OPERATIONS IN C (ARRAY OF CHARACTERS)

```C/C++
#include <stdio.h>
```

```C/C++
#include <string.h>
```

```C/C++
int main() {
```

```C/C++
    // Declaration and initialization of a character array
(string)
```

```C/C++
    char greeting[] = "Hello, World!";
```

```C/C++
    // Printing the original string
```

```C/C++
    printf("Original String: %s\n", greeting);
```

```C/C++
    // Finding the length of the string
```

```C/C++
    size_t length = strlen(greeting);
```

```C/C++
    printf("Length of the string: %zu\n", length);
```

```C/C++
    // Copying the string to another array
```

```C/C++
    char copy[20];
```

```C/C++
    strcpy(copy, greeting);
```

```C/C++
    printf("Copied String: %s\n", copy);
```

```C/C++
    // Concatenating two strings
```

```C/C++
    strcat(copy, " How are you?");
```

```C/C++
    printf("Concatenated String: %s\n", copy);
```

```C/C++
    return 0;
```

```C/C++
}
```

ARRAYS IN FUNCTIONS

Key Points:

Arrays decay into pointers.

Size information is not automatically passed.

Passing arrays of different sizes.

Arrays are **not passed by value** (they are passed by reference).

Example 1: Printing Array

```C/C++
void printArray(int *arr, int size) {
```

```C/C++
    for (int i = 0; i < size; ++i) {
```

```C/C++
        printf("%d ", *(arr + i));
```

```C/C++
    }
```

```C/C++
    printf("\n");
```

```C/C++
}
```

Example 2: Modifying Array

```C/C++
void modifyArray(int arr[]) {
```

```C/C++
    arr[0] = 100;
```

```C/C++
}
```

```C/C++
int main() {
```

```C/C++
    int myArray[3] = {1, 2, 3};
```

```C/C++
    modifyArray(myArray);   // array is modified
```

```C/C++
    return 0;
```

```C/C++
}
```

COMMON PITFALLS

Buffer Overflows

Uninitialized Arrays

Off-by-One Errors

Mixing up row and column indices in 2D arrays

Confusing array names with pointers

BEST PRACTICES

Use named constants

Check array bounds

Initialize arrays properly

Use `sizeof` for array sizes

Understand pointer arithmetic

REAL-WORLD EXAMPLES OF ARRAYS

Databases

Graphics & Image Processing

Matrices in Mathematics

Signal Processing

Weather Data Analysis

Financial Applications

Simulation and Modelling

Network Routing